

DL065 设备与性能实验

本实验要交付一份能说明“测了什么、没有测什么”的小型性能报告。你会先检验两个实现的计算一致性，再运行真实训练、计时、状态字节计数与分析器。完成顺序很重要：如果两个程序计算的目标不同，比较它们的速度就失去了原本的问题。

1 准备环境与文件

推荐在独立Python3.11或3.12环境中安装requirements.txt。实验依赖PyTorch、NumPy与绘图工具；无需外部账户、预训练模型、网络数据或GPU。首次安装依赖通常需要网络，安装完后实验可离线运行。交付结果实际使用Python3.12.14与PyTorch2.14.1+cpu，完整版本在verification.json。

```
python -m pip install -r requirements.txt
python -m unittest -v test_experiment.py
python -O -m unittest -v test_experiment.py
python experiment.py --output outputs
python make_figures.py
```

请在本单元目录执行。默认输出会覆盖同名结果；扩展实验请用例如--output my_run另存。只从你信任的来源加载pt文件，示例采用weights_only=True。文档构建工具属于可选依赖，阅读PDF不需要重建环境。

2 先做纸笔账本

不运行程序，先写出32到128到128到2网络每层参数数。检查偏置是否计入。把总元素数乘4得到FP32参数字节；再估计参数、梯度、Adam两个状态的主要载荷。最后说明这份估计遗漏了哪些对象。

然后打开make_model与memory_ledger。前者使用三个Linear与两个ReLU；后者分别遍历参数、现存梯度和优化器state张量。创建Adam不一定立即创建全部状态，很多状态在第一次step时才懒初始化。这个行为应通过程序观察，而不是仅凭公式猜。

```
import torch
import experiment as e
m = e.make_model()
o = torch.optim.Adam(m.parameters(), foreach=False)
print(e.memory_ledger(m, o))
x, y = e.make_data(8)
e.step(m, o, x, y)
print(e.memory_ledger(m, o))
```

预期参数为20994个元素；训练前梯度与Adam状态为0，训练一步后梯度与参数载荷相同，Adam张量状态略超过两份参数。若版本导致状态布局不同，先逐项打印state，而不是强行修改结果以迎合预期。

3 验证向量化的语义

阅读step中的两个分支。逐行分支对每行保留长度为1的批维，然后拼接logits；它并没有对每行独立做一次优化器更新。两种方式都在整个批上计算一个平均交叉熵，最后更新一次。

11项测试包含数据形状、独立测试种子、参数数、状态字节、逐行与整批梯度/更新一致性、错误参数和训练下降等。测试使用unittest的检查方法及torch.testing.assert_close，优化模式下也不会像普通assert语句一样被删除。运行普通和-O两次，检查两次都确实执行了11项。

故障植入。 在自己的副本中，把逐行分支改为对每行分别调用optimizer.step。预测参数是否仍相等，然后运行测试。记录第一次不满足的条件。这个修改同时改变更新频率与优化轨迹，不能仅把它称为“另一种实现”。恢复代码后继续。

4 跑一份真实学习任务

experiment.py生成1024个训练例与512个独立测试例，用CPU单线程训练20个epoch。测试数据只用于报告，不用于挑选超参数或挑选最佳epoch。outputs/data.npz保留实际数组，trained_model.pt保留训练权重，results.json保留每个epoch结果。

本次终点准确率0.92578125。重新加载checkpoint，独立算一次准确率，并与JSON比较：

```
from pathlib import Path
import numpy as np
import torch
import experiment as e
m = e.make_model()
saved = torch.load('outputs/trained_model.pt', weights_only=True)
m.load_state_dict(saved['state_dict'])
a = np.load('outputs/data.npz')
with torch.no_grad():
    logits = m(torch.from_numpy(a['test_x']))
    score = (logits.argmax(1) == torch.from_numpy(a['test_y'])).float().mean()
print(float(score))
```

如果结果不一致，检查模型结构、dtype、数据文件及权重来源。测试集独立并不意味着一次测试就能证明现实有效性；当前问题是一个固定合成机制。

5 阅读实际计时记录

measure先创建该测量的模型与Adam，单独运行一步，预热5步，再记录7组、每组10步。先确认每组不是只包含前向；zero_grad、反向和Adam均在边界中。数据生成、模型构造、打印和文件保存均在边界外。

在results.json找到每个seconds_per_step_blocks，把秒乘1000得到毫秒。手工求中位数，再用64除以秒/step得到样本/秒。与保存字段核对。中位数与四分位数是观测摘要；不要把7组值看成7台独立机器的样本。

第一步记录不等于整台机器冷启动：当前进程已导入PyTorch且可能运行过其他模型。请把这一点写在报告。再次运行时计时变化是正常现象；测试不设置“必须加速几倍”的易碎阈值。

6 分析器与批量扫描

打开profiler.txt。找出self CPU时间最大的三个条目，并回到代码找对应计算或record_function范围。再比较total CPU列，解释为什么父子范围的total不能全部相加。self memory为负的条目可能发生了释放；不要把事件净变化当成峰值。

批量16、64、256的扫描在batch_sweep里。画吞吐曲线后提出一个可检验的瓶颈假设。若想判断数据读取是否主导真实任务，必须另做包含数据加载的端到端版本；当前计时没有这个环节。

分析器与稳态基准分开运行。记录形状和内存会引入开销，profiler中某个step的时间不应直接替换基准中位数。预热之前分配的对象也不一定能在后来的内存事件记录中完整回溯。

7 GPU扩展与诚实的空结果

有可信CUDA环境时，可以运行python experiment.py --device cuda --output gpu_run。代码在计时块前后同步，稳态内存测量前重置峰值计数并在测量后同步。报告设备型号、CUDA与PyTorch版本、精度、批量，以及allocated与reserved的区别。

本课没有实际执行GPU分支。当前结果中的gpu_memory为null，表示未测。不要从CPU逻辑字节数推算一个“实测显存”，也不要将null改成0。扩展分支的峰值范围包含该进程中所有仍存活的CUDA张量；若要只比较单个模型，应在全新进程隔离每个配置，保持对象生存期一致。

8 Notebook与重建

experiment.ipynb已在全新Python进程的真实IPython进程内核按顺序执行。它含公式手算、测试、实际完整重跑、训练图与计时图，PNG以Notebook输出内嵌。重跑写入notebook_outputs，避免覆盖教材固定outputs。这个验证不等于测试过浏览器Jupyter界面或外进程通信。

```
python create_notebook.py
python execute_notebook.py experiment.ipynb
python build_pdf.py lecture.md
python build_pdf.py lab.md
python build_pdf.py answers.md
```

9 练习与提交

- A. 对1000到100线性层算参数、梯度、两个Adam状态的主要字节；转换为MiB。
- B. 一个块20个step，批量128，用时0.08秒。求延迟、吞吐。若批量256时块用时0.12秒，解释两个指标分别如何变化。
- C. 原程序60%时间不可改变，剩余40%加速4倍。求整体加速比；说明理论上限。
- D. 写出一次不公平的性能比较：至少包含一个计时边界问题和一个学习目标变化。再给出修正协议。
- E. 设计一个达到目标验证误差的用时实验。预先指定目标、种子、最大预算和失败处理；既报告成功用时，也保留没有达到目标的运行。

提交代码改动、普通/-O测试输出、原始7组计时、账本作用域、checkpoint复核与一页结论。结论必须包括一个本实验不能回答的问题。逐步答案见answers.pdf，先完成自己的推导再核对。